

SnowPro Specialty: Snowpark — Prep Guide

⚠️ **Verify against the official exam guide as of July 2026** — Snowflake revises exam codes, domains, and weightings periodically. Confirm current details at snowflake.com/certifications before your exam.

PJ's Academy · Prove you can build data & ML pipelines with Snowpark.

The Snowpark Specialty exam (SPS-C01) validates deep skills with the **Snowpark** developer framework — DataFrames, UDFs/UDTFs, stored procedures, and ML workloads in Python/Java/Scala.

Prerequisite: SnowPro Core certified.

Exam At A Glance

Item	Detail
Exam code	SPS-C01
Questions	~55
Duration	85 minutes
Cost	\$225 USD
Prerequisite	SnowPro Core

Domain Weightings

#	Domain	Weight
1	Snowpark Fundamentals & Sessions	20%
2	DataFrame API & Transformations	30%
3	UDFs, UDTFs & Stored Procedures	30%
4	Performance, Deployment & ML	20%

Domain 1 — Fundamentals & Sessions (20%)

- Snowpark architecture — client builds a **lazy** plan; execution is pushed to Snowflake.
- Creating a `Session` (connection params, key-pair auth, `Session.builder`).
- Snowpark vs pandas vs SQL — when to use each.
- Anaconda package channel, imports, and stage-based dependencies.

```
from snowflake.snowpark import Session
session = Session.builder.configs(conn).create()
```

Domain 2 — DataFrame API & Transformations (30%)

- **Lazy evaluation** — transformations build a plan; actions (`collect` , `show` , `count` , `to_pandas`) trigger execution.
- Core ops: `select` , `filter` , `with_column` , `group_by` , `agg` , `join` , `sort` , `union` , `pivot` .
- Functions module: `col` , `lit` , `when` , window functions, `call_udf` .
- `cache_result` , `explain()` , saving with `write.save_as_table` (modes: overwrite/append).

```
from snowflake.snowpark.functions import col, avg, when
result = (session.table("sales")
         .filter(col("amount") > 1000)
         .with_column("tier", when(col("amount") > 5000, "high").otherwise("mid"))
         .group_by("region").agg(avg("amount").alias("avg_amt")))
result.show() # action → executes in Snowflake
```

Domain 3 — UDFs, UDTFs & Stored Procedures (30%)

- **Scalar UDFs** — return one value per row (`@udf`).
- **Vectorized UDFs** — process pandas batches (`@udf` + Pandas types) → faster.
- **UDTFs** — return a table (multiple rows) per input.
- **Stored procedures** (`@sproc`) — procedural logic, can run DDL/DML & orchestrate.
- Permanent vs temporary; `stage_location` , `packages` , `imports` .

```
from snowflake.snowpark.functions import udf
from snowflake.snowpark.types import IntegerType

@udf(name="add_gst", is_permanent=True, stage_location="@udf_stage",
     packages=["snowflake-snowpark-python"], replace=True)
def add_gst(amount: float) -> float:
    return amount * 1.18
```

Domain 4 — Performance, Deployment & ML (20%)

- Pushdown optimization; avoiding unnecessary `to_pandas` (pulls data to client).
- Vectorized UDFs & batching for throughput.
- Deploying with Snowpark: UDFs/sprocs to stages, CI/CD.
- **Snowpark ML** (`snowflake.ml.modeling`) — distributed training + Model Registry.
- Snowpark-optimized warehouses for memory-heavy ML.

High-Yield Facts

- Snowpark DataFrames are **lazy** — nothing runs until an **action** (`show` , `collect` , `count` , `to_pandas`).
- `to_pandas()` **pulls data to the client** — avoid on large data.
- **Vectorized (Pandas) UDFs** process batches → much faster than scalar row-by-row.
- **UDTF** returns multiple rows; **UDF** returns one value.
- **Stored procedures** can do DDL/DML and orchestrate; UDFs cannot.

- **Snowpark-optimized warehouses** provide more memory for heavy ML.
 - Packages come from the **Snowflake Anaconda channel**.
-

Practice Questions (10)

- Q1.** Snowpark DataFrame transformations are: A) Eager B) Lazy (run on an action) C) Cached always D) Client-side
- Q2.** Which triggers execution? A) filter B) with_column C) collect D) select
- Q3.** Which returns multiple rows per input? A) Scalar UDF B) UDTF C) Stored procedure D) View
- Q4.** Fastest for high-throughput Python inference: A) Scalar UDF B) Vectorized (Pandas) UDF C) UDTF D) External function
- Q5.** Which can run DDL/DML and orchestrate logic? A) UDF B) Stored procedure C) UDTF D) View
- Q6.** `to_pandas()` on a huge DataFrame is risky because it: A) Is lazy B) Pulls all data to the client C) Deletes data D) Requires a UDF
- Q7.** Memory-heavy ML training benefits from: A) XS warehouse B) Snowpark-optimized warehouse C) Multi-cluster D) Result cache
- Q8.** Python packages for Snowpark come from: A) PyPI directly B) Snowflake Anaconda channel C) GitHub D) Conda-forge only
- Q9.** Distributed model training in Snowpark uses: A) sklearn locally B) snowflake.ml.modeling C) UDTF D) Streams
- Q10.** A permanent UDF requires a: A) Warehouse B) stage_location C) Stream D) Task

Answers

1-B · 2-C · 3-B · 4-B · 5-B · 6-B · 7-B · 8-B · 9-B · 10-B

2-Week Plan

- **Week 1:** Sessions + DataFrame API (build real transformations).
 - **Week 2:** UDFs/UDTFs/sprocs + Snowpark ML + practice.
-

 *Snowflake Mastery — PJ's Academy*
